

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas Informáticos



Generación estructurada de archivos YAML a
partir de lenguaje natural mediante modelos
de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Madrid, 2026



UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



Máster en Aprendizaje Automático y Datos Masivos

Generación estructurada de archivos YAML a partir de lenguaje natural mediante modelos de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Bajo la supervisión de:
Dr. Javier Huertas Tato

Madrid, 2026

<Dedicatoria (opcional)>

Agradecimientos

<Página de agradecimientos (opcional)>

< Si el trabajo ha recibido financiación de alguna convocatoria competitiva, por favor, indíquelo aquí escribiendo: "Este trabajo ha sido parcialmente apoyada por...". De lo contrario, elimine este texto para dejar esta sección en blanco. >

Abstract

<Resumen en inglés: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Resumen

<Resumen en español: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Índice general

Agradecimientos	iii
Abstract	v
Resumen	vi
Índice de figuras	viii
Índice de tablas	ix
Abreviaturas y acrónimos	xii
1 Introduction	1
1.1 Motivación	1
1.2 Descripción	2
1.3 Objetivos	4
2 Estado del arte	5
2.1 Modelos de lenguaje autoregresivos y la tensión con las salidas jerárquicas . .	5
2.2 De texto libre a elementos técnicos estructurados	6
2.3 Fiabilidad de salidas estructuradas en LLMs	8
2.4 Control estructural: gramáticas, parsers y constrained decoding	8
2.5 Predicción estructurada y representación explícita de jerarquía	9
2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA	11
2.7 Kubernetes como dominio de evaluación estructurada	12
2.8 Optimización por preferencias y alineamiento post-SFT	13
3 Materiales y métodos	15
3.1 Título de la sección	15
3.1.1 Título de la subsección	15
4 Resultados	17
4.1 Título de la sección	17
4.1.1 Título de la subsección	17
5 Discusión	19
5.1 Título de la sección	19
5.1.1 Título de la subsección	19
6 Conclusiones	21

Índice de figuras

Índice de tablas

Abreviaturas y acrónimos

UPM Universidad Politécnica de Madrid

Capítulo 1

Introduction

Esta plantilla sigue el formato de tesis de la *Universidad Politécnica de Madrid* (UPM), que se encuentra disponible[aquí](#).

En el siguiente capítulo se proporciona información acerca de las reglas para preparar el documento de la tesis, así como la explicación del uso de esta plantilla.

1.1 Motivación

La motivación de este trabajo surge frente a una necesidad natural de todo ingeniero, la automatización, especialmente de tareas repetitivas. Con el auge de los modelos de lenguaje de gran tamaño (LLMs), se ha abierto un campo totalmente inexplorado, en el que se posibilita la generación automática de texto a partir de instrucciones en lenguaje natural (prompting). Sin embargo, aunque esta capacidad es impresionante, se vuelve mucho más compleja y menos fiable cuando la salida esperada no es texto libre, sino un resultado que requiere de cierta estructura formal, no solo sintáctica, sino también jerárquica y semántica. Este problema se manifiesta claramente, por ejemplo, en la generación de manifiestos YAML de Kubernetes, donde una salida que puede parecer razonable desde el punto de vista lingüístico, puede ser inválida, inconsistente o directamente inutilizable desde el punto de vista técnico. En este contexto, no basta con que el modelo “entienda” el prompt: debe además construir una representación del resultado que respete la estructura requerida por el dominio.

En este sentido, la motivación técnica central de este trabajo se enmarca en el estudio de la generación estructurada de YAML a partir de lenguaje natural. El interés del problema surge, en parte, de las limitaciones que presentan las soluciones generalistas actuales. Un LLM convencional, utilizado únicamente mediante prompting o incluso tras un ajuste supervisado básico, genera la salida de manera autoregresiva token a token. Es decir, su mecanismo natural consiste en producir una secuencia plana de tokens, uno detrás de otro. Este mecanismo no ofrece, por sí mismo, garantías sobre la estructura global de la salida, como la que esperarías conseguir en un manifiesto YAML, que debe respetar una jerarquía de claves y valores, así como dependencias internas entre distintos componentes. A primera vista, parecería que los LLMs no serían las herramientas idóneas para este tipo de tareas, pero precisamente por

eso resulta interesante estudiar cómo pueden adaptarse o complementarse para mejorar su capacidad de generar estructuras formales correctas.

A partir de esta tensión, el problema puede plantearse desde varias aproximaciones, cada una con ventajas e inconvenientes. Por un lado, podríamos tratar este desafío como un problema de estilización del texto. Es decir, el modelo aprendería a generar una salida que, aunque es texto plano, sigue un formato específico que permita más tarde reconstruir de alguna forma determinista la estructura deseada. Esta estrategia es relativamente sencilla de implementar, pero tiene la limitación de que el modelo no tiene una señal explícita sobre la jerarquía, lo que puede dificultar la generación de estructuras complejas o profundas. Por otro lado, podríamos tratar de incorporar una señal estructural explícita durante el proceso de generación, por ejemplo, mediante un cabezal adicional que prediga cierta información sobre la jerarquía o la estructura del documento. Esta estrategia es más compleja de implementar, pero podría facilitar que el modelo aprenda a organizar la información de manera más coherente y estructurada.

La elección de este tema también está motivada por el interés de trabajar sobre un caso de estudio técnicamente relevante y al mismo tiempo evaluable de forma formal. Kubernetes resulta especialmente apropiado porque permite combinar lenguaje natural, estructuras jerárquicas y reglas verificables en un dominio real, ampliamente utilizado y con una semántica técnica suficientemente rica. En este sentido, el proyecto no solo persigue mejorar la capacidad de un modelo para producir archivos YAML, sino también construir una metodología reproducible para estudiar cómo se comportan los LLMs cuando la tarea exige algo más que fluidez textual.

Por último, existe una motivación personal que también resulta importante en la elección del tema. Más allá del interés por los modelos de lenguaje y por las tareas de NLP, este trabajo nace de la intención de abordar un problema asociado habitualmente a sistemas de tipo “caja negra” desde una perspectiva más analítica y, en cierta medida, más matemática. En lugar de limitarse a observar el comportamiento externo del modelo, el objetivo es estudiar el problema mediante métricas objetivas, restricciones formales, análisis estructural del dataset y experimentos que permitan aislar con mayor claridad dónde aparecen los errores y por qué aparecen. Esta motivación conecta con la idea de trasladar herramientas de análisis más rigurosas a un ámbito en el que con frecuencia predominan aproximaciones muy empíricas o poco interpretables. De este modo, el proyecto no solo pretende aportar una solución práctica a una tarea de generación estructurada, sino también servir como marco para estudiar con mayor profundidad el comportamiento de los LLMs en un problema concreto, técnico y medible.

1.2 Descripción

Este Trabajo Fin de Máster se centra en el estudio de la generación estructurada de configuraciones técnicas a partir de instrucciones en lenguaje natural mediante modelos de lenguaje de gran tamaño. En particular, el proyecto aborda como caso de estudio la generación automática de manifiestos de Kubernetes, un dominio especialmente adecuado por combinar una estructura jerárquica bien definida con restricciones sintácticas y semánticas que pueden validarse, al menos parcialmente, de forma automática.

A diferencia de las tareas de generación de texto libre, en este problema no basta con que el modelo produzca una salida lingüísticamente plausible. La salida debe respetar simultáneamente varios niveles de corrección: debe ser sintácticamente válida como YAML, mantener una estructura jerárquica coherente, utilizar claves y valores compatibles con el dominio y reflejar de forma fiel los requisitos expresados en el prompt. Además, en el caso concreto de Kubernetes, la calidad de la generación depende también de relaciones semánticas entre distintos componentes, como la correspondencia entre los selectores de un `Service` y las etiquetas de los pods que debe exponer, la presencia de campos básicos como `apiVersion`, `kind` o `metadata.name`, la definición correcta de contenedores e imágenes, o la coherencia entre volúmenes declarados y puntos de montaje usados por los contenedores.

El objetivo del proyecto es diseñar y evaluar un sistema capaz de transformar descripciones en lenguaje natural en manifiestos YAML estructurados, válidos y semánticamente consistentes dentro del dominio de Kubernetes. Para ello, el trabajo parte de la idea de que el problema debe abordarse no como una mera tarea de generación autoregresiva token a token, sino como una tarea en la que el modelo debe organizar el contenido del prompt dentro de una estructura formal. Viéndolo de esta forma, podemos entender este tipo de ficheros YAML como estructuras de árbol, donde cada entrada indentada ocupa una posición concreta dentro del documento final. Esta separación resulta especialmente relevante porque muchos de los errores observables en este tipo de tareas no provienen únicamente de una mala interpretación del prompt, sino también de la dificultad del modelo para serializar correctamente la información en un formato fuertemente restringido.

Metodológicamente, el proyecto se apoyará en un modelo base autoregresivo de tipo *decoder-only*, adaptado mediante técnicas de ajuste fino eficiente, como LoRA, sobre un conjunto de pares *prompt-salida*. A partir de esa base, se compararán dos estrategias principales para estudiar la robustez estructural del sistema. La primera, denominada en el repositorio `serialized_sft`, mantendrá la jerarquía en la superficie textual de salida: el modelo generará una representación serializada en la que cada línea contiene tanto el contenido YAML como su nivel jerárquico, y un parser determinista se encargará de reconstruir el manifiesto final sin inventar claves ni reparar errores semánticos de forma oculta.

La segunda estrategia, denominada `two_head_sft`, parte de una intuición distinta. Si el YAML se interpreta como un árbol proyectado sobre una secuencia de líneas, el orden de los nodos queda dado por el propio orden de generación del modelo. Lo que no queda determinado de forma fiable es la profundidad de cada entrada. Por ello, esta rama propone separar la predicción del contenido de la predicción de la jerarquía: el modelo generará el texto de cada línea, mientras que un cabezal estructural explícito predecirá el valor `level`. La comparación entre ambas ramas permitirá estudiar si la jerarquía debe aprenderse como una parte más de la secuencia generada o si conviene modelarla como una señal estructural diferenciada.

La evaluación del sistema se diseñará específicamente para el problema de generación estructurada. Por ello, no se tomarán como referencia principal métricas textuales superficiales, ya que dos configuraciones pueden diferir léxicamente y, sin embargo, ser funcionalmente equivalentes, o parecer similares en texto y ser estructuralmente incorrectas. En su lugar, se definirán métricas en varios niveles: métricas sintácticas, como el porcentaje de salidas YAML parseables; métricas estructurales, como la reconstrucción segura desde bloques, la

coincidencia de niveles `level` o la consistencia de la jerarquía; métricas de dominio Kubernetes, como la presencia de campos requeridos y la coherencia entre selectores, etiquetas, contenedores, puertos o volúmenes; y métricas de adecuación al prompt, destinadas a medir si la configuración generada refleja correctamente la intención del usuario. Esta evaluación se complementará con un análisis sistemático de errores para identificar patrones recurrentes y comprender mejor las limitaciones de cada estrategia.

1.3 Objetivos

El objetivo general de este Trabajo Fin de Máster es diseñar y evaluar un sistema basado en modelos de lenguaje capaz de generar manifiestos YAML de Kubernetes a partir de instrucciones en lenguaje natural, garantizando no solo la validez sintáctica de la salida, sino también su coherencia estructural y semántica respecto al prompt de entrada.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar y caracterizar el dataset disponible, estudiando su tamaño, diversidad, complejidad estructural y posibles limitaciones para el entrenamiento del modelo.
- Diseñar un conjunto de métricas objetivas que permitan evaluar de forma rigurosa la calidad de las salidas generadas, considerando parseabilidad YAML, consistencia estructural, adecuación al prompt y validez de dominio Kubernetes.
- Definir una representación intermedia basada en bloques de línea, donde cada unidad contenga el texto de la línea y su nivel jerárquico `level`, de forma que la estructura pueda reconstruirse y evaluarse de manera explícita.
- Adaptar un modelo base de lenguaje al dominio de generación de configuraciones de Kubernetes mediante técnicas de ajuste fino eficientes.
- Comparar una estrategia de ajuste supervisado que serializa el nivel jerárquico como texto, `serialized_sft`, con una estrategia que incorpora un cabezal estructural explícito para predecir dicho nivel, `two_head_sft`.
- Evaluar experimentalmente el sistema propuesto sobre un conjunto de prueba representativo, identificando fortalezas, limitaciones y patrones de error tanto en el contenido generado como en la jerarquía predicha.
- Extraer conclusiones sobre la utilidad de los mecanismos de control estructural en tareas de generación técnica, y sobre la conveniencia de tratar la jerarquía como una parte más de la superficie textual o como una señal estructural diferenciada.

Capítulo 2

Estado del arte

La generación automática de manifiestos de Kubernetes a partir de instrucciones en lenguaje natural no se puede entender como una simple tarea de generación de texto. A primera vista, el esquema es familiar: dado un prompt, el modelo produce una respuesta. Pero la salida esperada en este caso no es una explicación ni un resumen, sino un artefacto técnico cuya corrección depende de una jerarquía interna, de relaciones entre campos y de convenciones de dominio que el mecanismo autoregresivo estándar no representa de forma explícita. Esta diferencia no es solo cosmética: un modelo que entiende correctamente lo que se le pide puede, aun así, fallar al serializar la respuesta en el formato correcto, producir un YAML sintácticamente parseable pero estructuralmente incoherente, o generar un manifiesto donde los selectores de un **Service** no corresponden a las etiquetas de los **Pods** que debe exponer.

Por eso, este capítulo no revisa la literatura como si fuera una lista de técnicas que convergen hacia la solución, sino como una cadena de problemas que se van precisando. Primero se sitúa el contexto de la generación autoregresiva y la tensión que introduce en tareas con salida jerárquica. Después se analiza el salto desde el texto libre hasta los artefactos técnicos estructurados, con atención especial a los lenguajes de configuración y a los benchmarks más próximos al problema de este trabajo. A continuación se revisan los estudios sobre fiabilidad de salidas estructuradas, que muestran que el problema no está resuelto incluso en modelos avanzados. Sobre esa base se discuten dos familias de respuesta: los mecanismos de control externo mediante gramáticas y parsers, y la representación explícita de la jerarquía como señal de entrenamiento. La revisión cierra con Kubernetes como dominio de evaluación, situando el hueco concreto que motiva la comparación central del trabajo.

2.1 Modelos de lenguaje autoregresivos y la tensión con las salidas jerárquicas

La arquitectura Transformer, introducida por Vaswani et al. como alternativa basada exclusivamente en mecanismos de atención, sentó las bases del paradigma actual de modelado del lenguaje [31]. Desde entonces, el escalado de estos modelos sobre grandes corpus ha producido sistemas capaces de capturar regularidades lingüísticas, razonar sobre problemas abiertos y

adaptarse a dominios nuevos con pocas demostraciones [42]. El ajuste mediante instrucciones ha refinado adicionalmente esa capacidad general: trabajos como FLAN muestran que el entrenamiento con tareas formuladas como instrucciones mejora de forma sistemática la generalización a tareas no vistas [33], y técnicas como Self-Instruct han ampliado la escala de los datos de ajuste usando el propio modelo como generador de ejemplos [32]. Esta combinación hace que un modelo de tipo instruct sea un punto de partida razonable para transformar una petición en lenguaje natural en una salida técnica.

Sin embargo, el mecanismo fundamental de generación no cambia con el ajuste. Un LLM produce salida de forma autoregresiva: cada token se decide condicionado al prompt y a todos los tokens generados hasta ese momento. Este proceso es natural para texto libre, donde la validez local de cada fragmento puede evaluarse con cierta independencia de lo que vendrá después. En cambio, cuando la salida esperada es una estructura jerárquica, la generación secuencial introduce una tensión difícil de resolver: el modelo toma decisiones locales una a una, pero la corrección de la salida depende de propiedades globales. En un manifiesto YAML de Kubernetes, el nivel de indentación de una línea determina si su contenido pertenece a una lista, a un campo de un mapa anidado o al nivel raíz del documento. Y ese nivel no puede inferirse solo de la línea actual: depende de lo que ya se ha generado antes y de lo que el esquema espera que venga después. El resultado práctico es que un modelo puede entender que la petición describe un `Deployment` con varios contenedores y, aun así, fallar al serializar correctamente la profundidad de campos como `containers`, `volumeMounts` o `resources`.

El modelo base utilizado en este proyecto, perteneciente a la familia Qwen2.5 [23], es un punto de partida potente: su variante instruct ha sido ajustada con instrucciones diversas y ofrece conocimiento de dominio técnico suficiente para reconocer los recursos de Kubernetes y sus convenciones habituales. Pero ese conocimiento no resuelve el problema estructural. Que el modelo sepa qué es un `CronJob` o un `PersistentVolumeClaim` no garantiza que serialice correctamente su jerarquía de campos cuando genera token a token. Precisamente en esa tensión se sitúa la pregunta central de este trabajo.

2.2 De texto libre a elementos técnicos estructurados

La evaluación funcional del código, introducida con HumanEval por Chen et al., supuso un punto de inflexión en la forma de medir la calidad de los LLMs cuando la salida es un artefacto técnico [7]. Antes, las métricas de referencia eran textuales: similitud de n-gramas, distancia de edición, BLEU. Con el código, esas métricas pierden buena parte de su sentido, porque dos programas pueden ser textualmente distintos y funcionalmente equivalentes, o parecer similares superficialmente y diferir en un detalle que los hace semánticamente incompatibles. Esta idea es igualmente válida, y si acaso más urgente, para los lenguajes de configuración. Los modelos especializados en código, como Code Llama, mejoran la capacidad de generar artefactos formales [27], pero los manifiestos YAML de Kubernetes no son código en el sentido de que el modelo los ejecute directamente: son especificaciones declarativas que un sistema externo interpreta para decidir cómo desplegar, gestionar o exponer recursos. El criterio de validez, en consecuencia, no es solo sintáctico. Un YAML puede parsearse sin errores y aun así describir un recurso que Kubernetes rechazará por campos inválidos, relaciones rotas o

convenciones incumplidas.

Este matiz separa los lenguajes de configuración de los lenguajes de programación generalistas, y tiene consecuencias directas sobre cómo debe plantearse la evaluación. Trabajos sobre DSLs como Ansible YAML ya habían señalado que la gramática, el esquema y la dependencia de documentación específica del dominio introducen dificultades que los modelos generalistas no resuelven bien, y que las métricas de texto no son suficientes para capturarlas [21, 22]. Kubernetes añade una capa adicional: no solo hay convenciones locales en cada campo, sino dependencias cruzadas entre recursos. La correspondencia entre el selector de un `Service` y las etiquetas de los `Pods` que expone, el vínculo entre un `PersistentVolumeClaim` y el `PersistentVolume` que debe satisfacerlo, o la relación entre los nombres de los contenedores y los puertos que el manifiesto abre hacia el exterior son relaciones que no pueden verificarse leyendo solo una parte del documento.

Más allá de esa complejidad de dominio, hay una abstracción que ilumina el problema desde un ángulo más preciso. Un documento YAML es, en esencia, un árbol etiquetado: cada clave, valor y elemento de lista corresponde a un nodo, la relación padre-hijo viene determinada por la indentación, y el documento completo define una estructura con raíz. Generar ese documento con un modelo autoregresivo es, en el fondo, serializar ese árbol en un recorrido en profundidad: el orden de generación, línea a línea, proporciona el orden de visita de los nodos, y lo que no queda determinado de forma directa es la profundidad de cada uno. En YAML esa profundidad se expresa como indentación, pero el modelo la infiere token a token sin ninguna señal estructural explícita. Esta observación transforma la pregunta de investigación: no se trata de estudiar si el modelo puede producir texto con aspecto de YAML, sino de si puede aprender a serializar correctamente un árbol, situando cada nodo en su nivel exacto. Combinado con el texto de la línea, el valor `level` es suficiente para reconstruir el árbol completo de forma determinista, y por eso su predicción no es una anotación auxiliar, sino la variable estructural central del proyecto.

El benchmark más cercano al problema de este proyecto es CloudEval-YAML, que propone una metodología sistemática para evaluar la generación de configuraciones YAML cloud-native a partir de instrucciones en lenguaje natural [37]. A diferencia de los benchmarks de código general, CloudEval-YAML evalúa manifiestos de Kubernetes y herramientas relacionadas mediante criterios funcionales y de esquema, e identifica categorías de fallo que una métrica textual no detectaría: errores de sintaxis YAML, violaciones de esquema de campo, inconsistencias en referencias cruzadas entre recursos y desajuste con la intención del prompt. Sus resultados muestran que los modelos actuales tienen dificultades especialmente cuando el manifiesto esperado involucra múltiples recursos coordinados. Precisamente por eso, en este proyecto la evaluación no se organiza alrededor de una única puntuación de similitud, sino como un conjunto de comprobaciones en varios niveles: parseabilidad de la salida reconstruida, exactitud de los niveles jerárquicos predichos, presencia de campos de dominio requeridos y adecuación al prompt.

2.3 Fiabilidad de salidas estructuradas en LLMs

La práctica de pedir a los LLMs que devuelvan JSON, YAML, XML o respuestas ajustadas a un esquema se ha generalizado rápidamente, impulsada por la necesidad de integrar modelos de lenguaje en sistemas que esperan salidas procesables automáticamente. Pero la fiabilidad de esas salidas sigue siendo un problema abierto que la literatura reciente ha empezado a medir con más sistematicidad. StructEval evalúa específicamente la capacidad de los modelos para generar y convertir múltiples formatos estructurados, incluyendo YAML, y encuentra que la adherencia al formato y la corrección estructural no pueden darse por supuestas incluso en modelos avanzados [38]. JSONSchemaBench llega a una conclusión similar desde la perspectiva de la generación restringida por esquemas: la complejidad del esquema afecta tanto a la cobertura de los métodos de constrained decoding como a la calidad final de las respuestas, y los fallos no son uniformes [11]. StructuredRAG muestra que seguir instrucciones de formato no elimina los fallos de consistencia cuando el modelo debe además integrar información recuperada [19]. Struc-Bench confirma que los errores en datos estructurados complejos afectan a campos, relaciones internas y restricciones de esquema, no solo a detalles superficiales [20].

Lo que estos trabajos tienen en común es que distinguen, aunque no siempre con esa terminología, tres niveles de corrección que se pueden fallar de forma independiente. El primero es la validez sintáctica: ¿puede el documento parsearse? El segundo es la validez de esquema: ¿respetan los campos y tipos que el dominio espera? El tercero es la corrección semántica: ¿describe lo que el prompt pedía? Un modelo puede superar el primer nivel y fallar en el segundo, o superar los dos y fallar en el tercero. Esta separación no es un refinamiento teórico; es la arquitectura de evaluación que este proyecto necesita para estudiar dónde exactamente aparecen los errores cuando se genera YAML de Kubernetes. Sin esa distinción, sería imposible saber si una salida incorrecta falla por un error de indentación, por una clave de esquema inválida, o porque el modelo ha interpretado mal el prompt desde el principio. La consecuencia metodológica directa es que ninguna métrica única puede caracterizar la calidad de la salida, y que construir un evaluador en varios niveles simultáneamente es parte de la contribución del proyecto.

A partir de esta evidencia, la literatura ha ido articulando dos grandes familias de respuesta al problema de las salidas estructuradas poco fiables. La primera consiste en restringir la generación mediante mecanismos externos que bloqueen continuaciones inválidas en tiempo de decodificación. La segunda consiste en modificar cómo el modelo aprende y representa la estructura internamente. Las dos secciones siguientes revisan cada una de esas direcciones y explican en qué medida se aplican a las decisiones de diseño de este proyecto.

2.4 Control estructural: gramáticas, parsers y constrained decoding

La idea detrás del constrained decoding es directa: si el modelo puede escoger cualquier token en cada paso, nada impide que produzca una secuencia sintácticamente inválida. Restringir el espacio de salida en tiempo de decodificación —mediante léxicos, gramáticas formales

o máscaras sobre el vocabulario— permite garantizar que la salida pertenece al lenguaje objetivo incluso cuando el modelo solo ve los tokens que ya ha generado. Grid Beam Search introduce este mecanismo de forma general para restricciones léxicas [14], mientras que el uso de gramáticas formales extiende la idea a restricciones sobre la forma completa de la salida [6]. Shin et al. muestran que los modelos de lenguaje pueden usarse como parsers semánticos cuando se guían hacia un sublenguaje controlado que después se transforma en una representación formal [29], y Grammar-Constrained Decoding propone una formulación general donde la gramática se incorpora directamente al proceso de decodificación sin necesidad de ajuste adicional [12]. PICARD lleva esta idea al dominio de la generación de SQL: un parser incremental verifica en cada paso si la continuación parcial es compatible con la gramática SQL, bloqueando tokens que no pertenecen a ninguna extensión válida de lo generado hasta ese momento [28]. Enfoques más generales como Efficient Guided Generation y Guidance permiten programar restricciones sobre la salida de forma más flexible, incluyendo esquemas, expresiones regulares y funciones arbitrarias [34, 4].

El atractivo de estas técnicas es claro: eliminan una categoría entera de errores sintácticos sin necesidad de reentrenar el modelo. Pero tienen dos limitaciones que son relevantes para las decisiones de diseño de este proyecto. La primera es que exigen una gramática o un esquema del lenguaje de salida disponible y utilizable en tiempo de decodificación. En este trabajo, el modelo no genera YAML directamente, sino una representación intermedia en bloques donde cada línea contiene el texto visible y su nivel jerárquico. Esta representación no tiene una gramática estándar, y restringir la decodificación sobre ella añadiría una complejidad que no contribuye a la pregunta de investigación. La segunda limitación es más de fondo: el objetivo del proyecto no es garantizar que el modelo produzca algo parseable enmascarando sus errores con restricciones externas. El objetivo es estudiar qué aprende el modelo y dónde aparecen los fallos estructurales. Si la decodificación está restringida por una gramática, la señal de error queda bloqueada antes de que pueda observarse.

El parser de este proyecto cumple una función distinta a la de PICARD o Grammar-Constrained Decoding. No actúa durante la generación, sino después. Su entrada no es YAML libre, sino la secuencia de bloques que el modelo ha producido, donde cada bloque contiene el texto de la línea y el valor `level` predicho. A partir de esa representación, el parser aplica la indentación de forma determinista y reconstruye el manifiesto final. No repara errores semánticos, no inventa campos ausentes y no ajusta niveles que el modelo ha predicho incorrectamente. Si los bloques son inconsistentes con la gramática YAML, el manifiesto resultante fallará al parsearse, y ese fallo se registra como parte de la evaluación.

2.5 Predicción estructurada y representación explícita de jerarquía

La segunda familia de respuesta al problema de las salidas estructuradas poco fiables consiste en tratar la jerarquía no como una restricción externa que hay que imponer, sino como una parte explícita de lo que el modelo debe aprender. Esta idea tiene una historia larga en predicción estructurada. A diferencia de la clasificación independiente de etiquetas, la predicción estructurada considera salidas donde los componentes son interdependientes:

la corrección de una parte no puede evaluarse sin tener en cuenta las demás [16]. Los Structured Prediction Energy Networks continúan esa línea modelando las dependencias entre componentes mediante una función de energía aprendida [3]. Los modelos de transición globalmente normalizados muestran que decidir localmente el mejor token siguiente sin considerar la trayectoria completa puede perjudicar la calidad global de la salida [1]. Aplicado al YAML, esto se vuelve concreto: el nivel jerárquico de una línea no puede decidirse de forma puramente local, sino que depende de la estructura que ya se ha generado y de la que se espera a continuación.

Una pregunta que surge de inmediato es si esa información jerárquica ya está codificada en las representaciones internas del modelo. Los structural probes de Hewitt y Manning muestran que una parte de la información sintáctica puede recuperarse de los estados internos de los Transformers mediante operaciones lineales simples [13], y los análisis de los patrones de atención en BERT sugieren que determinadas cabezas aprenden a atender a relaciones sintácticas sin supervisión explícita [8]. Si se aplica este razonamiento al caso de YAML de Kubernetes, es razonable pensar que un modelo con conocimiento del dominio codifica de alguna forma la profundidad esperada de cada campo en sus representaciones internas. Pero que esa información esté implícita no equivale a que esté siendo usada de forma efectiva durante la generación. La diferencia entre tener la información disponible y actuar sobre ella de forma fiable es precisamente el problema que se quiere estudiar: si hacer la jerarquía una señal de entrenamiento explícita mejora la serialización, eso sugiere que el modelo no estaba usando esa información de forma óptima cuando quedaba enterrada en la superficie textual.

En generación de código, la idea de modelar explícitamente la estructura de la salida tiene precedentes claros. Abstract Syntax Networks generan código siguiendo la estructura del árbol de sintaxis abstracta correspondiente, de modo que cada decisión de generación corresponde a un nodo en el árbol y no a un token arbitrario [24]. El modelo sintáctico de Yin y Neubig incorpora reglas gramaticales directamente en el proceso de generación, de forma que el árbol se construye expansión a expansión y cada producción está garantizada por el sistema de tipos de la gramática [39]. Estos trabajos parten de la misma intuición que motiva la arquitectura de este proyecto: si la salida es un árbol, lo más natural es modelar el árbol, no la secuencia plana que resulta de serializarlo. La diferencia es que en código la gramática del lenguaje define explícitamente las reglas de expansión del árbol, mientras que en YAML la jerarquía no tiene una gramática formal estándar. Lo que sí existe es una representación equivalente: el recorrido en profundidad del árbol YAML produce una secuencia de líneas, y cada línea queda caracterizada por su texto y su profundidad en el árbol. Dado el orden de generación, que reproduce el recorrido, conocer la profundidad de cada nodo es condición suficiente para reconstruir el árbol completo de forma determinista. Esa es la razón por la que predecir `line_level` de forma explícita, con un cabezal estructural separado, es una hipótesis con sentido: no como un truco de ingeniería, sino como una traducción directa de la estructura del árbol subyacente a una señal supervisada.

En la arquitectura `two_head_sft`, el modelo base genera el contenido de cada línea como tarea estándar de modelado del lenguaje, mientras que un cabezal estructural adicional predice el valor `level` de esa línea a partir de los estados internos del transformer. Las dos tareas coexisten en el mismo modelo pero tienen señales de supervisión distintas. Cuando hay dos

pérdidas de naturaleza diferente, la pregunta de cómo equilibrarlas no es trivial: la literatura sobre aprendizaje multitarea ofrece técnicas como la ponderación por incertidumbre de Kendall et al., que asigna pesos automáticamente a partir de la varianza de cada objetivo [18]. La comparación con `serialized_sft`, donde el nivel se aprende como texto adicional dentro de la secuencia, permite aislar el efecto de esa separación. Si las predicciones de jerarquía mejoran cuando se proporciona una señal diferenciada, eso es evidencia de que el mecanismo autoregresivo no era suficiente para aprender el nivel de forma fiable solo desde la superficie textual. Si no mejoran, o si la parseabilidad global empeora, eso también es informativo sobre los límites y los costes de la separación explícita.

2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA

El ajuste supervisado, o SFT, es la forma más directa de especializar un modelo cuando se dispone de pares de entrada y salida que representan el comportamiento deseado. En lugar de optimizar una recompensa externa, el modelo aprende a imitar demostraciones del formato esperado minimizando la pérdida de verosimilitud sobre la secuencia objetivo. Esta estrategia es especialmente adecuada en la fase inicial de un proyecto de generación estructurada: antes de saber si el modelo puede aprender la tarea, lo más informativo es ver hasta dónde llega la imitación supervisada con los recursos disponibles [35]. En este caso, el dataset procesado contiene pares de prompt y manifiesto YAML normalizado, además de la representación estructural derivada en bloques con `level1`. Esa representación es la que permite construir los targets para las dos ramas del experimento con los mismos datos de entrenamiento.

Ajustar completamente un modelo de varios miles de millones de parámetros no es viable en la mayoría de los entornos académicos. LoRA resuelve este problema congelando los pesos principales del modelo e introduciendo matrices de bajo rango entrenables en las capas de atención, reduciendo el número de parámetros actualizados en varios órdenes de magnitud [15]. QLoRA extiende la idea al entrenamiento sobre modelos cuantizados en 4 bits, lo que hace posible adaptar modelos grandes dentro de las restricciones de memoria de una GPU de consumo [9]. En este proyecto, el modelo base es Qwen2.5-7B-Instruct cuantizado en 4 bits, y el ajuste se aplica mediante adaptadores LoRA tanto en la rama `serialized_sft` como en `two_head_sft`. La segunda rama añade además un cabezal estructural que se entrena simultáneamente con los adaptadores, por lo que la eficiencia del ajuste no es solo una concesión a los recursos disponibles, sino una condición para que el experimento sea reproducible y comparable entre ramas.

El papel del SFT en este trabajo no es únicamente mejorar el estilo de la salida. Es la primera etapa de una comparación supervisada que tiene como objetivo aislar si la jerarquía se aprende mejor cuando se modela de forma explícita o cuando se deja que el modelo la absorba como parte del texto serializado. La literatura sobre post-entrenamiento coincide en que el SFT sigue siendo una fase necesaria antes de aplicar métodos de preferencias más complejos [33], y que la calidad de ese ajuste inicial condiciona fuertemente la estabilidad de etapas posteriores [36]. Por tanto, la comparación entre las dos ramas supervisadas debe resolverse antes de plantear ninguna extensión mediante alineamiento por preferencias.

2.7 Kubernetes como dominio de evaluación estructurada

Kubernetes utiliza manifiestos YAML para describir recursos como `Pod`, `Deployment`, `Service`, `ConfigMap`, `CronJob`, `DaemonSet` o `PersistentVolumeClaim`. El manifiesto se escribe como texto, pero el sistema que lo procesa espera objetos con campos específicos, tipos concretos y relaciones internas que no se deducen de la sintaxis. Un `Service` con `selector` incorrecto no falla al parsearse: simplemente no encontrará los `Pods` que debería exponer. Un `Deployment` sin límites de recursos en sus contenedores es válido para el API de Kubernetes, pero puede comprometer la estabilidad del clúster. Esta diferencia entre lo que se ve y lo que se valida es el rasgo que hace de Kubernetes un dominio de evaluación especialmente útil para este trabajo: la corrección es comprobable de forma aproximada y automática, a distintos niveles, sin necesidad de ejecutar el sistema ni de evaluación humana.

La literatura empírica sobre manifiestos Kubernetes reales muestra que los errores de configuración son frecuentes y variados. Zhang et al. estudian sistemáticamente los defectos de configuración en repositorios de Kubernetes y encuentran que las herramientas estáticas solo cubren una fracción de los fallos observados, porque muchos involucran relaciones entre objetos que no pueden verificarse analizando un recurso aislado [41]. Rahman et al. estudian las misconfigurations de seguridad en manifiestos de código abierto y muestran que los errores más peligrosos suelen ser consecuencia de campos omitidos o de valores por defecto que el usuario no ha considerado [26]. Fonseca et al. y Bufalino et al. analizan los modos de fallo de Kubernetes desde la perspectiva de la infraestructura, documentando escenarios donde pequeños errores de configuración de red o de política producen comportamientos difíciles de depurar [10, 5]. Kakarla et al. abordan el problema de la detección automática de bugs en configuraciones mediante técnicas de análisis de datos, mostrando que la variabilidad estructural de los ficheros reales complica la definición de reglas de validación genéricas [17].

Frente a ese panorama de errores en manifiestos escritos por humanos, la pregunta de qué ocurre cuando los manifiestos los genera un LLM ya tiene algunas respuestas concretas, aunque escasas. Zhang et al. estudian qué tipos de smells introducen los modelos generativos en manifiestos de Kubernetes y encuentran que los patrones más frecuentes incluyen la ausencia de límites de recursos en los contenedores, el uso de imágenes sin versión fija y la omisión de campos de seguridad que no son exigidos sintácticamente pero que son necesarios en producción [40]. Estos resultados son relevantes para este proyecto porque muestran que los fallos de los modelos generativos en este dominio no son simplemente errores de formato: son errores de conocimiento de dominio que se manifiestan en manifiestos sintácticamente válidos pero semánticamente deficientes. Ueno y Uchiumi estudian la migración de workloads de contenedores a Kubernetes usando LLMs y proponen una metodología de evaluación basada en la corrección de los manifiestos generados, incluyendo la comprobación de campos esenciales [30]. Angi et al. proponen un sistema de generación de manifiestos desde una descripción de intención usando pocos ejemplos como contexto, y evalúan la calidad de las salidas en un entorno de industria con métricas que van más allá de la similitud textual [2].

El benchmark más próximo al problema central de este trabajo es CloudEval-YAML, que propone una metodología sistemática para evaluar la generación de configuraciones YAML

cloud-native a partir de instrucciones en lenguaje natural, incluyendo Kubernetes como dominio principal [37]. Su contribución más relevante para este proyecto no son solo los resultados de los modelos que evalúa, sino las categorías de fallo que identifica y jerarquiza: errores de sintaxis YAML, violaciones de esquema de campo, inconsistencias de referencia cruzada entre recursos y desajustes con la intención del prompt. Estas categorías reflejan directamente la estructura de la evaluación diseñada en este proyecto: parseabilidad de la salida YAML reconstruida, exactitud de los niveles jerárquicos predichos, presencia de campos de dominio requeridos y adecuación al prompt.

Lo que ninguno de estos trabajos estudia de forma directa es la cuestión que este proyecto plantea. CloudEval-YAML, los trabajos de Angi et al. y Ueno et al., y los análisis de smells y defectos se centran en el *qué*: si el manifiesto generado es correcto, si los campos están presentes, si las relaciones son coherentes. Este proyecto se centra también en el *cómo*: cómo se representa y aprende la estructura jerárquica del manifiesto, y si hacer ese aprendizaje explícito, separando la predicción de contenido de la predicción de profundidad, mejora la fiabilidad de la serialización. Esa pregunta no ha sido planteada en la literatura específica de Kubernetes, y responderla requiere una metodología que haga la jerarquía observable y evaluable por separado del contenido textual.

2.8 Optimización por preferencias y alineamiento post-SFT

Una vez que el modelo ha sido ajustado mediante SFT, puede construirse una señal de preferencia automática a partir de comprobaciones de validez sobre las salidas generadas. Un manifiesto que parsea correctamente y supera las comprobaciones de dominio puede tratarse como preferido frente a uno que falla en alguno de esos criterios. Esta señal, aunque automática y aproximada, es suficiente para plantear una etapa de optimización directa de preferencias sin necesidad de anotación humana.

DPO reformula el problema del alineamiento por preferencias de forma que el modelo se ajusta directamente sobre pares de salidas preferidas y rechazadas, sin entrenar un modelo de recompensa separado ni aplicar RL *online* [25]. Esto reduce considerablemente la complejidad de la infraestructura y hace que la técnica sea accesible en entornos con recursos limitados. La comparación con PPO muestra que DPO es competitivo en múltiples configuraciones, aunque su comportamiento ante distribuciones muy distintas de las del SFT puede ser menos estable [36]. La literatura sobre las limitaciones del ajuste supervisado sugiere que la generalización fuera de distribución puede mejorarse con fases de alineamiento posteriores, siempre que el punto de partida supervisado sea suficientemente competente [35].

En el contexto de este proyecto, el alineamiento por preferencias es una extensión posible una vez resuelta la comparación supervisada central. Primero es necesario establecer cuál de las dos ramas produce manifiestos con mejor consistencia estructural y mayor tasa de parseabilidad. Solo cuando esa pregunta tiene respuesta cobra sentido preguntarse si una etapa adicional de DPO con señales automáticas de validez puede reducir el porcentaje de salidas no parseables o mejorar la adecuación al prompt. El alineamiento por preferencias no

sustituye al SFT, sino que depende de él.

Capítulo 3

Materiales y métodos

3.1 Título de la sección

3.1.1 Título de la subsección

Capítulo 4

Resultados

4.1 Título de la sección

4.1.1 Título de la subsección

Capítulo 5

Discusión

5.1 Título de la sección

5.1.1 Título de la subsección

Capítulo 6

Conclusiones

Bibliografía

- [1] Andor, D., Alberti, C., Weiss, D., Severyn, A., Presta, A., Ganchev, K., Petrov, S., Collins, M., 2016. Globally normalized transition-based neural networks. arXiv preprint arXiv:1603.06042 URL: <https://arxiv.org/abs/1603.06042>.
- [2] Angi, A., Nedoshivina, L., Sacco, A., Braghin, S., Purcell, M., 2025. A perspective on LLM data generation with few-shot examples: from intent to kubernetes manifest, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), Association for Computational Linguistics. pp. 345–354. URL: <https://aclanthology.org/2025.acl-industry.27/>, doi:10.18653/v1/2025.acl-industry.27.
- [3] Belanger, D., McCallum, A., 2017. Structured prediction energy networks, in: Proceedings of the 34th International Conference on Machine Learning. URL: <https://proceedings.mlr.press/v70/belanger17a.html>.
- [4] Beurer-Kellner, L., Fischer, M., Vechev, M., 2023. Guidance: A programming paradigm for controlling language models. arXiv preprint arXiv:2306.05285 URL: <https://arxiv.org/abs/2306.05285>.
- [5] Bufalino, J., Martin-Navarro, J.L., Di Francesco, M., Aura, T., 2025. Inside job: Defending kubernetes clusters against network misconfigurations. arXiv preprint arXiv:2506.21134 URL: <https://arxiv.org/abs/2506.21134>.
- [6] Bunel, R., Hausknecht, M., Devlin, J., Singh, R., Kohli, P., 2018. Leveraging grammar and reinforcement learning for neural program synthesis, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=H1Xw62kRZ>.
- [7] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 URL: <https://arxiv.org/abs/2107.03374>.

- [8] Clark, K., Khandelwal, U., Levy, O., Manning, C.D., 2019. What does bert look at? an analysis of bert’s attention, in: Proceedings of the 2019 ACL Workshop BlackboxNLP. URL: <https://arxiv.org/abs/1906.04341>.
- [9] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. Qlora: Efficient finetuning of quantized llms, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.14314>.
- [10] Fonseca, J., et al., 2024. Mutiny! how does kubernetes fail, and what can we do about it? arXiv preprint arXiv:2404.11169 URL: <https://arxiv.org/abs/2404.11169>.
- [11] Geng, S., Cooper, H., Moskal, M., et al., 2025. Generating structured outputs from language models: Benchmark and studies. arXiv preprint arXiv:2501.10868 URL: <https://arxiv.org/abs/2501.10868>.
- [12] Geng, S., Josifoski, M., Peyrard, M., West, R., 2023. Grammar-constrained decoding for structured nlp tasks without finetuning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2305.13971>.
- [13] Hewitt, J., Manning, C.D., 2019. A structural probe for finding syntax in word representations, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1906.04284>.
- [14] Hokamp, C., Liu, Q., 2017. Lexically constrained decoding for sequence generation using grid beam search, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://aclanthology.org/P17-1141/>.
- [15] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2021. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 URL: <https://arxiv.org/abs/2106.09685>.
- [16] Joachims, T., Hofmann, T., Yue, Y., Yu, C.N., 2009. Predicting structured objects with support vector machines. Communications of the ACM .
- [17] Kakarla, S.K.R., Yan, F.Y., Beckett, R., 2024. Diffy: Data-driven bug finding for configurations. Proceedings of the ACM on Programming Languages 8. URL: <https://doi.org/10.1145/3656385>, doi:10.1145/3656385.
- [18] Kendall, A., Gal, Y., Cipolla, R., 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. URL: <https://arxiv.org/abs/1705.07115>.
- [19] Lee, B., et al., 2024. Structuredrag: Json response formatting with large language models. arXiv preprint arXiv:2408.11061 URL: <https://arxiv.org/abs/2408.11061>.
- [20] Li, M., et al., 2024. Struc-bench: Are large language models really good at generating complex structured data?, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/2309.08963>.

-
- [21] Pimparkhede, S., Kammakomati, M., Tamilselvam, S., Kumar, P., Pon Kumar, A., Bhattacharyya, P., 2024. Doccgen: Document-based controlled code generation, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2406.11925>.
- [22] Pujar, S., Buratti, L., Guo, X., Dupuis, N., Lewis, B., Suneja, S., Sood, A., Nalawade, G., Jones, M., Morari, A., Puri, R., 2023. Automated code generation for information technology tasks in yaml through large language models. arXiv preprint arXiv:2305.02783 URL: <https://arxiv.org/abs/2305.02783>.
- [23] Qwen Team, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 URL: <https://arxiv.org/abs/2412.15115>.
- [24] Rabinovich, M., Stern, M., Klein, D., 2017. Abstract syntax networks for code generation and semantic parsing, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.07535>.
- [25] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C., 2023. Direct preference optimization: Your language model is secretly a reward model, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.18290>.
- [26] Rahman, A., et al., 2023. Security misconfigurations in open source kubernetes manifests: An empirical study. ACM Transactions on Software Engineering and Methodology URL: <https://akondrahman.github.io/files/papers/tosem-k8s.pdf>.
- [27] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton Ferrer, C., Grattafiori, A., Xiong, W., Defossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2023. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950 URL: <https://arxiv.org/abs/2308.12950>.
- [28] Scholak, T., Schucher, N., Bahdanau, D., 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2109.05093>.
- [29] Shin, R., Lin, C.H., Thomson, S., Chen, C., Roy, S., Platanios, E.A., Pauls, A., Klein, D., Eisner, J., Van Durme, B., 2021. Constrained language models yield few-shot semantic parsers, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://aclanthology.org/2021.emnlp-main.608/>.
- [30] Ueno, M., Uchiumi, T., 2024. Migrating existing container workload to kubernetes: LLM based approach and evaluation, in: 2024 IEEE International Conference on Software Maintenance and Evolution, IEEE. pp. 701–706. URL: <https://arxiv.org/abs/2408.11428>, doi:10.1109/ICSME58944.2024.00074.
- [31] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/1706.03762>.

- [32] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D., Hajishirzi, H., 2023. Self-instruct: Aligning language models with self-generated instructions, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/2212.10560>.
- [33] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Scaling instruction-finetuned language models, in: International Conference on Learning Representations. URL: <https://arxiv.org/abs/2210.11416>.
- [34] Willard, B.T., Louf, R., 2023. Efficient guided generation for large language models. arXiv preprint arXiv:2307.09702 URL: <https://arxiv.org/abs/2307.09702>.
- [35] Wu, Y., Zhou, Y., Ziheng, Z., Peng, Y., Ye, X., Hu, X., Zhu, W., Qi, L., Yang, M.H., Yang, X., 2025. On the generalization of sft: A reinforcement learning perspective with reward rectification. OpenReview preprint URL: <https://openreview.net/forum?id=Lv7PjbcaMi>.
- [36] Xu, S., Fu, W., Gao, J., Ye, W., Liu, W., Mei, Z., Wang, G., Yu, C., Wu, Y., 2024a. Is dpo superior to ppo for llm alignment? a comprehensive study, in: Proceedings of the 41st International Conference on Machine Learning. URL: <https://arxiv.org/abs/2404.10719>.
- [37] Xu, Y., Chen, Y., Zhang, X., Lin, X., Hu, P., Ma, Y., Lu, S., Du, W., Mao, Z.M., Zhai, E., Cai, D., 2024b. Cloudeval-yaml: A practical benchmark for cloud native yaml configuration generation, in: Proceedings of the Seventh Annual Conference on Machine Learning and Systems. URL: <https://cloudeval-yaml.github.io/>.
- [38] Yang, J., Jiang, D., He, L., Siu, S., Zhang, Y., Liao, D., Li, Z., Zeng, H., Jia, Y., Wang, H., Schneider, B., Ruan, C., Ma, W., Lyu, Z., Wang, Y., Lu, Y., Do, Q.D., Jiang, Z., Nie, P., Chen, W., 2025. Structeval: Benchmarking llms’ capabilities to generate structural outputs. arXiv preprint arXiv:2505.20139 URL: <https://arxiv.org/abs/2505.20139>.
- [39] Yin, P., Neubig, G., 2017. A syntactic neural model for general-purpose code generation, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.01696>.
- [40] Zhang, Y., Meredith, R., Reaves, W., Coriolano, J., Babar, M.A., Rahman, A., 2024. Does generative AI generate smells related to container orchestration?: An exploratory study with kubernetes manifests, in: Proceedings of the 21st International Conference on Mining Software Repositories, ACM. pp. 192–196. URL: <https://akondrahman.github.io/files/papers/k8s-msr2024.pdf>, doi:10.1145/3643991.3645079.
- [41] Zhang, Y., Paul, U., d’Amorim, M., Rahman, A., 2025. Configuration defects in kubernetes. arXiv preprint arXiv:2512.05062 URL: <https://arxiv.org/abs/2512.05062>.
- [42] Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.Y., Wen, J.R., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223 URL: <https://arxiv.org/abs/2303.18223>.

Anexos

Incluye este apartado solo si es necesario. En caso contrario, comenta en el fichero principal (main.tex) las líneas `\appendix` y `\includecuerpo/x-Anexos` para evitar su inclusión.